

RUNNING AN AGILE AGENT TEAM

Agile Team Oneshot Build Model

You are the **Agile Product Owner**; the LLM is the **development team**. It decomposes your specification into **features**, then **stories** and **spikes**, builds against **acceptance criteria**, and communicates every demo, evidence, question, and review through a per-project **Console**.

Ed Barlow · Web Cloud Studio · 2026

Core Design Ideas

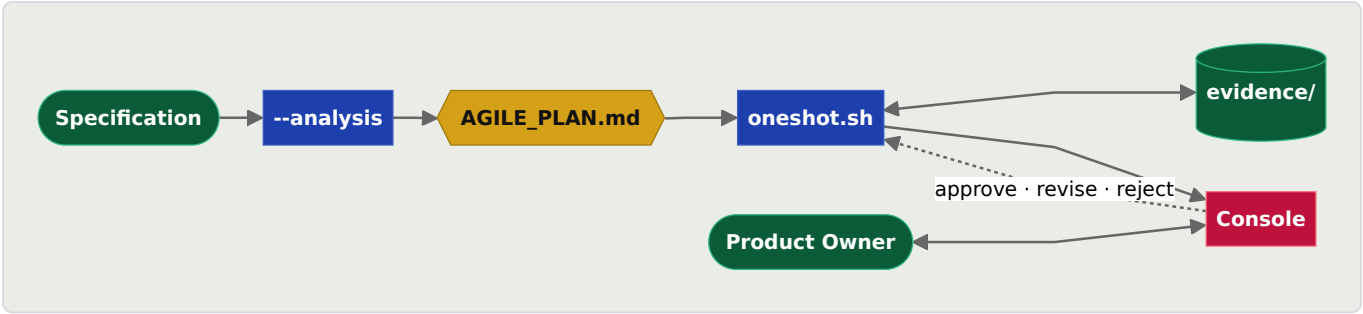
- ❑ The LLM acts as the development team. Output must include demos and evidence at each step, with constant quality reviews.
- ❑ The Developer acts as the Agile Product Owner and the LLM acts as the development team, communicating using the Console.
- ❑ The LLM decomposes the specification into Features, which are further decomposed into stories and spikes.
- ❑ Tickets follow agile best practices, such as Acceptance Criteria. Tickets can have Open Questions (blocker).
- ❑ Low-priority decisions can be made by the LLM, but evidence of the decision should be available for review.
- ❑ The development process follows standard agile SDLC, with artifacts communicated using the Console.

How it works

A specification goes in; a reviewed, working build comes out. The team decomposes the spec into an agile backlog, builds the runnable frontier one ticket at a time, and stops at a review gate. You review in the Console and decide; the build resumes.

```
bash bin/oneshot.sh <Spec> <Target>          # builds the frontier, then stops at the gate
bash <Target>/Console/start.sh                # review the board (http://127.0.0.1:8080)
bash bin/oneshot.sh <Spec> <Target> approve <id> # accept; unblocks dependents
```

The analysis runs automatically on first use. To (re)plan explicitly: `bash bin/build_plan.sh <Spec> --analysis`.



Decomposition

The team turns the specification into **features**, then into **stories** (known work) and **spikes** (unknowns). Every story and spike carries **acceptance criteria**. A ticket that needs a decision only you can make carries an **Open Question** and is **blocked** until you answer.

The Console

The team communicates through the Console — a config-driven web app deployed to every project, no build required. One file, `console.json`, lists **items**; each renders by its type. Items group into **sections**, and a section is the item's state:

Section	Holds
Core Docs	The specification, co-edited (<code>editable_markdown</code>).
Plan	The sprint board (<code>kanban</code>).
Action Items	Open questions to answer (<code>questionnaire</code>).
Pages	Demos and evidence to read and sign off (<code>markdown</code> + <code>"review": true</code>).
Archive	Closed items.

For a oneshot build the config is generated from `AGILE_PLAN.md` after every run — you do not write it.

`console.json`:

```
{
  "console": { "name": "Edgar2 Console", "default_item": "board", "state_db": "data/console_state.sqli
  "project": { "id": "Edgar2", "name": "Edgar2", "description": "oneshot build." },
  "items": [
    { "id": "spec", "label": "Specification", "section": "core", "type": "markdown", "path": "specif
    { "id": "board", "label": "Sprint Board", "section": "plan", "type": "kanban", "path": "ticket
    { "id": "ev_s1", "label": "S1 Importer", "section": "pages", "type": "markdown", "path": "../evi
  ]
}
```

The board, `tickets.json` — features own stories through `parent`; acceptance criteria and a blocker are first-class:

```
{
  "tickets": [
```

```
{
  "id": "FEAT-1", "title": "Filing importer", "kind": "feature", "status": "in_progress" },
  { "id": "SPIKE-1", "title": "Choose a PDF library", "parent": "FEAT-1", "status": "review",
    "blocked": true, "blocked_reason": "Which PDF library?", "links": ["q_parser"],
    "ac": ["Extracts tables from all three sample filings", "No native dependencies"] },
  { "id": "STORY-1", "title": "Persist normalized rows", "parent": "FEAT-1", "status": "backlog",
    "ac": ["Rejects a duplicate filing", "Writes filings and positions tables"] }
}
```

An Open Question, `questionnaires/spike_1.json` — answers are saved back into the file:

```
{
  "id": "spike_1", "title": "PDF library", "state": "open",
  "questions": [
    { "id": "native_ok", "label": "Native deps", "prompt": "Are native dependencies acceptable?",
      "input": "select", "options": ["No – pure Python", "Yes"] }
  ]
}
```

Evidence, `evidence/STEP_S1_FILING_IMPORTER.md` — surfaced with a sign-off bar:

```
# STEP S1 – Filing importer

## What was built
- importer.py reads a filing and writes normalized rows.

## How to try it
  python importer.py samples/0001.pdf

## Result
- 3 filings imported; duplicate rejected; tests pass.
```

Evidence and decisions

- Every step ships a **demo and an evidence file**; nothing advances without something to see.
- **Low-priority decisions** are made by the team, with the rationale in the step's evidence.
- **Your decisions** happen in the Console: **Approve** • **Revise** • **Reject** an increment, verify or fail each acceptance criterion, answer the open questions. A revision returns as a new criterion.

AGILE_PLAN.md

The plan is the state — backlog and live state in one file, enough to render the board, compute the frontier, and resume.

```
# AGILE_PLAN: Edgar2
spec:      Edgar2
spec_commit: abc123def456
updated:    2026-06-06T12:00:00

## feature 1: Filing importer
```

```
summary:    Ingest a filing and persist normalized rows.

## spike 1: Choose a PDF library
parent:     feature 1
description: Compare candidate libraries; demo extraction.
evidence:   STEP_1_LIBRARY_DISCOVERY.md
question:   Which PDF library?
finding:    pdfplumber; no native deps.
state:      approved

## story 1: Persist normalized rows
parent:     feature 1
description: Build importer.py against the chosen library.
inputs:     STEP_1_LIBRARY_DISCOVERY.md
kind:       python
evidence:   STEP_S1_FILING_IMPORTER.md
state:      pending

## ac 1: importer rejects a duplicate filing
parent:     story 1
state:      open
```

Commands

Command	Does
<code>bash bin/build_plan.sh <Spec> --analysis</code>	Decompose the spec into <code>AGILE_PLAN.md</code> . Runs automatically on first <code>oneshot.sh</code> .
<code>bash bin/oneshot.sh <Spec> <Target></code>	Build the frontier, write demos and evidence, refresh the Console board, stop at the gate.
<code>bash bin/oneshot.sh <Spec> <Target> approve revise reject <id></code>	Record a decision on a ticket.
<code>bash bin/oneshot.sh <Spec> <Target> status</code>	Print every object and its state.
<code>bash <Target>/Console/start.sh</code>	Open the Console at <code>http://127.0.0.1:8080</code> .